Curious Jack

A User Guide

Paul Fishwick and Claude Code

March 2026

Contents

1 Deep Looking

What do you see when you experience reality? Your experience is captured by an image or perhaps multiple images in a video. Within these artifacts, one can use multiple lenses for greater insight into looking. We define a *lens* as a category or element of knowledge classification (classes you took in school, or phrases in a Dewey Decimal system or Library of Congress). We call this **Deep Looking**. As you go about your day, take time to reflect on what you are seeing and ask questions. Many questions and answers to questions are rooted in academic subjects (disciplines). Look through the References for further information on related material. Deep Looking can be done anywhere and for any human experience. Museums are training wheels since they have collections of unique objects with deep stories.

2 Introduction

Curious Jack is a web application that uses multiple AI models to answer questions about an image. A user uploads a photograph or artwork, selects educational parameters, and types one or more questions. The application sends each question — together with the image — to three large language models (Google Gemini, OpenAI GPT, and Anthropic Claude) running in parallel via the OpenRouter API. A coordinator model then synthesizes the three responses into a single, level-appropriate answer and aligns it to the relevant educational standards for the chosen US state.

3 Getting Started

3.1 Starting the Server

Curious Jack runs as a local Node.js server. From the project directory, run the included start script:

```
./start
```

The server starts on `http://localhost:3456`. Open that URL in your browser to use the application.

To stop the server, run:

```
./stop
```

4 API Key Configuration

Curious Jack requires an OpenRouter API key to call the language models. There are two ways to provide it.

4.1 Environment Variable (Recommended)

Set `OPENROUTER_API_KEY` in your terminal before starting the server, or add it permanently to `~/.env`:

```
# Temporary (current terminal session only)
export OPENROUTER_API_KEY=sk-or-...
./start
```

```
# Permanent (add to ~/.env)
OPENROUTER_API_KEY=sk-or-...
```

When the key is found at startup, the application skips the modal entirely and the key never touches the browser or local storage — the most secure option.

4.2 In-Browser Modal

If no environment variable is set, the application checks the browser's `localStorage` for a previously saved key. If none is found, the API Key modal appears, as shown in Figure 1. Enter your OpenRouter key and click Save & Continue. The key is saved to `localStorage` so it persists across page reloads.

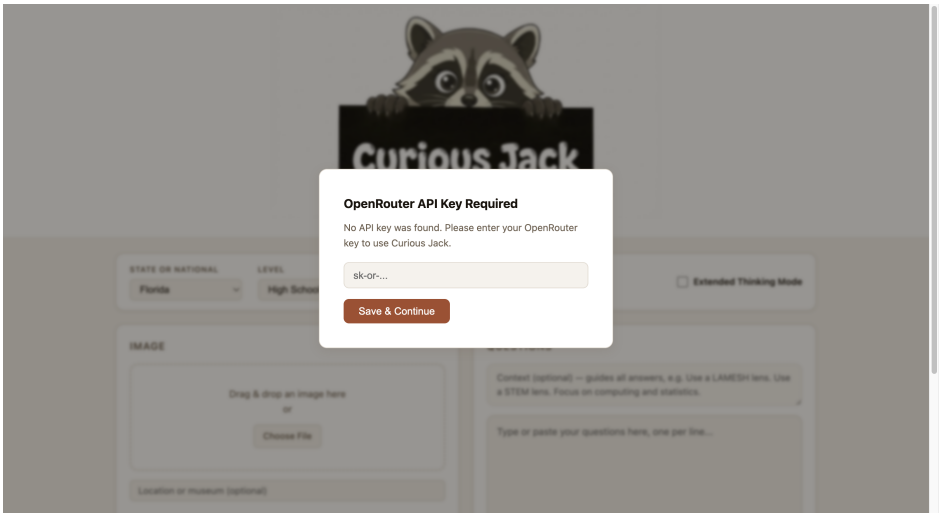

Figure 1: The API key modal appears on first use when no server-side key is configured. The key is stored in `localStorage` for subsequent visits.

5 The Interface

The main interface, shown in Figure 2, consists of a parameters bar at the top and two input cards below it — one for the image and one for the questions.

5.1 Parameters Bar

Four parameters shape the AI response:

- **State** — the US state whose educational standards are cited in the results (e.g. Florida, Texas, National).
- **Level** — the academic level of the answer: Elementary, Middle, High School, or Advanced / College.
- **Detail** — Low (one subject, concise), Medium (two or three subjects), or High (four or more subjects, comprehensive).
- **Read Aloud** — when checked, each answer is automatically read aloud via text-to-speech after it is rendered. The audio is generated by OpenAI's `gpt-4o-audio-preview` model

Figure 2: The main Curious Jack interface. The parameters bar runs across the top; the image upload card is on the left and the questions card is on the right.

through OpenRouter. A progress label shows "Processing audio for question N..." while the audio is being generated.

- **Voice** — selects the TTS voice used for Read Aloud. Six voices are available: Alloy, Echo, Fable, Onyx, Nova, and Shimmer.
- **Extended Thinking Mode** — when checked, each model uses its extended reasoning capability (Claude uses `thinking` blocks, Gemini uses the `:thinking` variant, GPT uses `reasoning_effort: high`).

5.2 Uploading an Image

Drag and drop an image onto the left card or click Choose File to open a file picker. Once loaded, a preview of the image replaces the drop zone. An optional Location or museum field lets you note where the image was taken or sourced; this context is passed to the models and included in the exported report.

5.2.1 Image Lightbox and Zoom Lens

Clicking the image preview opens a full-screen lightbox. A slow-looking prompt is displayed beneath the image: *"Take your time and look carefully at this. After a minute, think about questions you have."* This encourages patient observation before posing questions to the AI.

Hovering over the lightbox image activates a circular zoom lens (loupe) that follows the cursor and shows a magnified region of the *original uploaded file* — not the downscaled screen version. The scroll wheel controls the zoom level from 0.5× to 8× of the original pixel resolution. Click anywhere or press Escape to close the lightbox.

5.3 Context

An optional Context field appears above the questions textarea. One or two sentences entered here are passed as a guiding prefix to all worker models and the coordinator for every question in the session. Context is useful for specifying a lens or interpretive framework, for example:

```
All answers should use a LAMESH lens.
Focus on the computing and statistics perspectives.
```

When the context mentions a recognised lens keyword, the server automatically appends the full lens definition from the `~/lenses/` directory into the prompt. The currently supported lenses are listed in Table 1.

Keyword	Lens	File
`LAMESH`, `STEAM`, `STEM`	LA MESH (Language, Art, Mathematics, Engineering, Science, History)	`lamesh.md`
`computing`	Computing / Computer Science	`computing.md`
`statistics`	Statistics	`statistics.md`

Table 1: Lens keywords that trigger automatic injection of lens definitions from `~/lenses/`.

5.4 Entering Questions

Type one question per line in the questions textarea below the context field. You may also click Load from file to populate the textarea from a `.txt` or `.md` file. Blank lines and Markdown headers are ignored, so you can paste a structured notes file directly. Figure 3 shows the application with an image loaded, context entered, and questions ready.

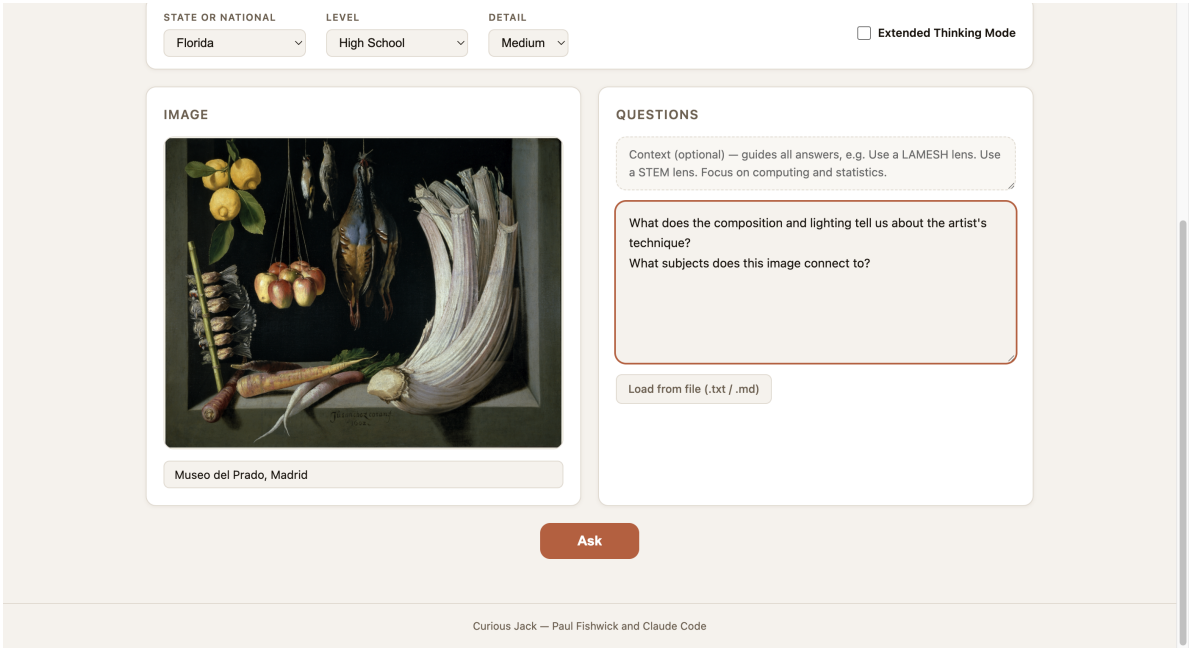

Figure 3: The application with an image uploaded and questions entered, ready to submit. The image preview replaces the drop zone and a location note has been added.

6 Results

Clicking Ask processes each question in turn. A progress bar and label track the current question number. As each question completes, its answer card appears and scrolls into view. A sample results card is shown in Figure 4.

Each card contains:

- The question number and the original question text.
- **Subject tags** — the academic disciplines identified by the coordinator model (e.g. Art History, Physics, Chemistry). Each tag is a clickable link to its Wikipedia article.
- **Answer** — a synthesized, markdown-formatted explanation that notes where the three worker models agreed and where they differed.
- **Educational Standards** — one to three standards from the chosen state that are directly relevant to the question, with standard codes and links to the official standards pages.

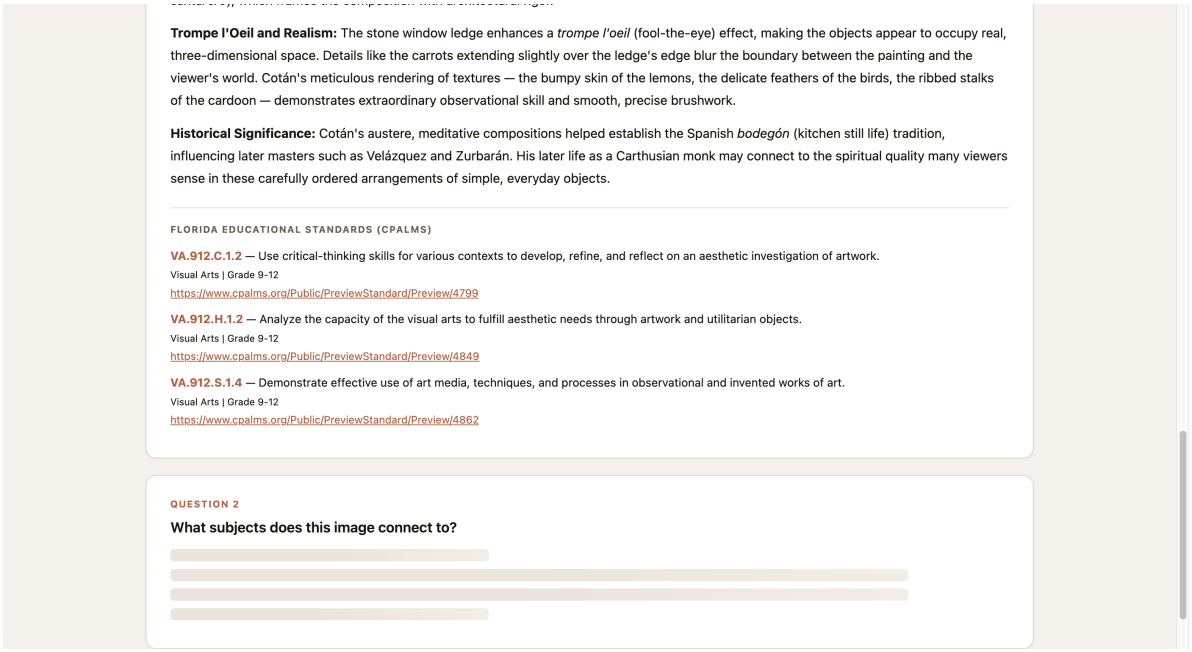

Figure 4: A completed answer card showing subject tags, the synthesized answer, and aligned Florida educational standards with CPALMS codes and links.

6.1 Text-to-Speech

Each answer card includes a speaker button () that reads the answer aloud on demand. Clicking the button sends the answer text (with Markdown formatting stripped) to the `/api/tts` server endpoint, which calls OpenAI's `gpt-4o-audio-preview` model via OpenRouter. The audio is streamed as PCM16 data, wrapped in a WAV header server-side, and played back in the browser.

When the Read Aloud checkbox in the parameters bar is checked, TTS plays automatically after each answer is rendered — the next question is not processed until playback completes. The voice can be changed at any time using the Voice dropdown; the new voice takes effect on the next TTS request.

Note that TTS generation involves an API call to generate the speech audio, so there is a delay of several seconds (longer for longer answers) before playback begins.

6.2 Multi-Model Synthesis

Internally, each question is sent to three worker models in parallel:

Worker	Model (via OpenRouter)
Gemini	`google/gemini-3.1-pro-preview`
OpenAI	`openai/gpt-5.2`
Claude	`anthropic/claude-opus-4-6`
Coordinator	`anthropic/claude-opus-4-6`

Table 2: The four model roles used per question. The coordinator synthesizes the three worker responses.

The coordinator is instructed to: state majority-agreed facts confidently, acknowledge genuine disagreements, incorporate unique insights from individual workers where plausible, and align the final answer to the chosen educational standards.

6.3 Image Generation

If a question asks the AI to *create* or *generate* an image (e.g. "Draw a diagram of. . . "), the coordinator routes the request to Google Gemini's image generation model (`gemini-3-pro-image-preview`) rather than returning a text answer. The generated image is displayed in the results card with a download link.

7 Export

After all questions are answered, two export buttons appear below the results:

- Export Markdown — downloads a `curiosity-report.md` file containing the full report: parameters, subject image (downscaled and embedded as an HTML `<img>` tag with base64 data), all questions, answers, and standards. The embedded image renders in markdown editors that support inline HTML, such as the companion Markdown Editor application.
- Export PDF — opens the browser's native print dialog. Select "Save as PDF" from the destination dropdown. The PDF includes the Curious Jack banner, title, parameters, the uploaded image, and all answer cards with standards.

Sessions are auto-saved to the server under `images/<label>/` where `<label>` is a short descriptive name generated by the coordinator model from the uploaded image. Each session folder contains:

- `image.<ext>` — a copy of the uploaded image
- `session.json` — all questions, answers, and parameters
- `report.md` — the exported markdown report (written on export)

8 Security

Because the API key can reside in the browser's `localStorage`, the frontend includes several layers of protection against cross-site scripting (XSS) attacks that could attempt to read and exfiltrate it.

8.1 DOMPurify Sanitization

AI model responses are converted from Markdown to HTML using the `marked` library and then injected into the DOM via `innerHTML`. A malicious response could embed `<script>` tags or `onerror` handlers designed to steal the stored key. All `marked.parse()` output is therefore passed through DOMPurify before touching the DOM:

```
DOMPurify.sanitize(marked.parse(answer.answer || ''))
```

DOMPurify strips dangerous tags and event attributes while preserving safe formatting.

8.2 Content Security Policy

A `Content-Security-Policy` meta tag instructs the browser to refuse any script not loaded from an approved source, and to block outbound connections to any host other than `localhost`:

```
default-src 'self';
script-src 'self' https://cdn.jsdelivr.net
           https://cdnjs.cloudflare.com;
style-src 'self' 'unsafe-inline';
img-src 'self' data: blob:;
connect-src 'self' https://cdnjs.cloudflare.com
            https://cdn.jsdelivr.net;
media-src 'self' blob:;
```

The `media-src blob:` directive permits playback of TTS audio blobs generated from the `/api/tts` endpoint.

Even if an XSS payload bypassed DOMPurify, the browser would block it from executing or phoning home.

8.3 HTML Escaping

All user-supplied text (questions, location field, subject tags, standards text) is passed through an `escHtml()` function before being inserted into `innerHTML` template strings. This prevents user-crafted input from being interpreted as HTML.

9 Technical Notes

- The server is a lightweight Express.js application (`server.js`) that proxies all model calls server-side, keeping the API key out of browser network traffic when using the environment-variable method.
- The frontend (`index.html` + `app.js`) is a static single-page application with no build step required.
- `dotenv` is configured to load `~/.env` automatically, so a key placed there is available to all projects that use the same pattern.

- The server runs on port **3456**.
- API endpoints are rate-limited to **30 requests per IP per 10 minutes** using `express-rate-limit`, protecting users who supply their own key from runaway usage.
- Lens definitions are loaded from `~/lenses/` at server startup and cached in memory.

10 References and Inspirations

The following works informed the design and pedagogical philosophy of Curious Jack — in particular the ideas of slow, attentive looking, patient observation, and curiosity-driven inquiry.

- **Slow Art Day** — slowartday.com. An annual global event in which participants spend extended time looking at a small number of works of art, followed by group discussion. Curious Jack encourages the same unhurried attention before submitting questions.
- Shari Tishman, ***Slow Looking***. Harvard Education Press. Tishman's research on slow, deliberate looking as a vehicle for deep learning across disciplines underpins the observation prompt shown in the image lightbox.
- Rob Walker, ***The Art of Noticing***. robwalker.net. A practical guide to paying closer attention to the world, with exercises for sharpening perception and curiosity — the spirit Curious Jack tries to cultivate through structured questioning.
- Alexandra Horowitz, ***On Looking: Eleven Walks with Expert Eyes***. alexandrahorowitz.net. Horowitz revisits a single city block with eleven different experts, showing how knowledge and attention transform what we see. The multi-model synthesis in Curious Jack echoes this multi-perspective approach to observation.
- ***New York Times* 10-Minute Challenge** — nytimes.com/spotlight/10-minute-challenge. A collection of short, timed exercises in close reading and observation. The lightbox prompt in Curious Jack ("Take your time and look carefully…") shares the same intention.
- **Tate Art Galleries** — tate.org.uk/art/guide-slow-looking. A practical guide to slow looking from one of the world's leading contemporary art institutions.